Case Study: Fastify Integration with Thendral e-Service

1. Project Background

Thendral e-Service is a platform with:

- **Two apps**:
    1. Customer app → sends service requests
    2. Shop app → receives notifications, accepts/rejects requests

Current backend: NestJS with **Express**.

Core workflow:

1. Customer enters location & selects service (e.g., aadhar, aadhar update).
2. System sends service request to shops within 5 km.
3. Shops can accept or reject. If rejected, the request goes to the next shop with a 30-second interval.
4. Timer stops when one shop accepts. Booking details sent to the customer.

Requirements for backend:

- Handle JSON request/response
- Validation of incoming requests
- Real-time notifications
- Fast response time
- Scalable for high traffic

---

2. What is Fastify

- **Fastify** is a modern Node.js web framework like Express but **highly optimized for speed**.
- It has **built-in JSON schema validation**, **low overhead**, and **high throughput** for REST APIs.
- Designed for both **small and large-scale applications**, including **serverless or microservices**.

Key Points:

- Can handle **30–50% more requests per second** compared to Express under typical load.
- Provides **hooks, plugins, and lifecycle events** for fine control.
- Automatic request validation if you define JSON schemas.

---

3. How Fastify Integrates with NestJS

1. **NestJS is adapter-based** → allows using **FastifyAdapter** instead of default Express.
2. Existing controllers, services, and modules mostly **remain unchanged**.
3. Routes in NestJS continue using decorators (@Controller, @Post(), @Get()).
4. Validation is still handled by **DTOs + ValidationPipe** in NestJS.
5. Fastify improves performance under the hood without requiring major code changes.

Integration Steps Overview:

- Replace Express adapter with Fastify adapter in main.ts.
- Existing NestJS controllers/services continue to work.

4. Benefits of Using Fastify for Thendral e-Service

Feature	Benefit for Thendral
Performance	Handles higher requests per second → useful if many customers send simultaneous service requests.
Automatic JSON Schema Validation	Prevents invalid booking requests from reaching services → reduces bugs and crashes.
Low overhead & fast serialization	Response to customers and shops is faster → improves user experience.
Plugin ecosystem	Plugins for authentication, JWT, CORS, file upload, multipart data → easier to extend features.
Serverless friendly	Can deploy functions quickly in cloud → scalable without heavy infrastructure.
Lifecycle hooks	Provides hooks before/after requests → can log, modify requests, or handle retries efficiently.

5. Differences from Express in NestJS

Aspect	NestJS + Express	NestJS + Fastify
Speed	Good for small to medium traffic	Faster under high load (30–50% more requests per second)
Request validation	DTO + ValidationPipe	DTO + ValidationPipe OR Fastify JSON schema
Response handling	Return JSON object	Return JSON OR Fastify reply object
Plugins	Express middleware (CORS, JWT)	Fastify plugins (CORS, JWT, multipart)
Serialization	Standard JSON serialization	Optimized JSON serialization for speed
Error handling	Standard NestJS error filters	Standard NestJS error filters + optional Fastify hooks

---

6. Useful Fastify Plugins for Thendral e-Service

Plugin	Purpose
fastify-cors	Handle cross-origin requests for mobile apps
fastify-jwt	Secure authentication for customers and shops
fastify-multipart	Handle uploads if shops upload service proof/photos
fastify-swagger	Auto-generate API documentation
fastify-rate-limit	Prevent abuse or accidental overload of requests
fastify-compress	Compress responses → reduce network latency
fastify-helmet	Security headers for safety

Note: NestJS has wrapper support for most of these, or you can register them at app-level.

7. When Fastify Makes Sense for Thendral e-Service

- **High traffic scenario**: Many customers sending requests simultaneously → Fastify reduces latency.
- **Critical speed**: Shops need near-instant notifications → faster response improves experience.
- **Automatic validation and performance**: Prevent invalid service requests without extra code.

---

8. When Express Might Be Enough

- Project has **low to medium traffic** (few hundred requests/sec).
- Team is **already familiar with Express middleware** and NestJS default setup.
- No strict performance bottlenecks → simpler to maintain.

---

9. Conclusion / Recommendation

- **Fastify + NestJS** is a **good fit for Thendral e-Service** if:
    - You anticipate **scaling the platform** with many simultaneous customer requests.
    - **Performance and speed** are critical for customer/shop experience.
    - You want **modern, structured validation** and fast JSON processing.
- **Express + NestJS** works for now, but may limit scalability if traffic grows quickly.